

AI Agent数据治理实战手册 - 完整代码合集

说明：以下代码均来自实战手册，按文件名分类，可直接复制保存为对应.py/.txt文件，修改配置后即可运行，无需额外调试。

一、基础配置文件（config.py）

```
1  # AI Agent数据治理基础配置文件（config.py）
2  # 直接复制保存为config.py，仅修改以下2处核心参数
3  class AIAgentDataGovernConfig:
4      # 1. 修改为你下载的开源大模型路径（解压后的文件夹路径）
5      LLM_MODEL_PATH = "D:/models/qwen-7b-chat" # Windows示例
6      # LLM_MODEL_PATH = "/home/user/models/llama-3-8b-instruct" # Linux示例
7
8      # 2. 修改为数据存储路径（用于存放原始数据、处理后数据）
9      DATA_STORAGE_PATH = "D:/ai_agent_data" # Windows示例
10     # DATA_STORAGE_PATH = "/home/user/ai_agent_data" # Linux示例
11
12     # 以下参数无需修改，直接套用
13     BATCH_PROCESS_SIZE = 5000 # 批量处理数据量（根据电脑配置调整）
14     CONTEXT_LENGTH = 2048 # 大模型上下文长度
15     DATA_QUALITY_TARGET = 0.99 # 数据治理目标准确率
16     MANUAL_INTERVENTION_RATE = 0.01 # 人工干预率目标（≤1%）
17
18     # 初始化配置，后续代码直接调用
19     config = AIAgentDataGovernConfig()
```

运行说明：1. 无需单独运行，是所有其他代码的依赖文件，必须先保存为config.py并放在同一目录下；2. 核心修改点为LLM_MODEL_PATH（大模型解压路径）和DATA_STORAGE_PATH（数据存储路径），需根据自身系统（Windows/Linux）选择对应示例修改；3. 其他参数可根据电脑配置和业务需求微调，新手建议默认不变。

二、环境校验代码（env_check.py）

```
1  # 环境校验代码（直接复制运行）
2  from config import config
3  from langchain.llms import LlamaCpp
4
5  try:
```

```

6      # 初始化大模型
7      llm = LlamaCpp(
8          model_path=config.LLM_MODEL_PATH,
9          n_ctx=config.CONTEXT_LENGTH,
10         temperature=0.1
11     )
12     # 测试大模型响应
13     test_response = llm("请简单介绍AI Agent在数据治理中的作用")
14     print("环境校验通过!")
15     print("大模型测试响应:", test_response[:50]) # 输出前50字符, 验证可用性
16 except Exception as e:
17     print(f"环境校验失败, 错误原因: {str(e)}")
18     print("请检查模型路径是否正确、依赖包是否安装完整")

```

运行说明：1. 保存为env_check.py，与config.py放在同一目录；2. 运行前需确保已安装所有依赖包（langchain、LlamaCpp等，安装指令见实战手册2.1节）；3. 核心作用是校验环境是否搭建成功，若输出“环境校验通过”，则可正常运行后续场景代码；4. 失败需优先检查config.py中的模型路径是否正确、依赖包是否安装完整。

三、场景1：结构化数据自动化清洗（clean_data_agent.py）

```

1  # 结构化数据自动化清洗完整代码 (clean_data_agent.py)
2  import pandas as pd
3  from langchain.llms import LlamaCpp
4  from langchain.prompts import PromptTemplate
5  from config import config
6
7  # 1. 初始化大模型 (无需修改)
8  llm = LlamaCpp(
9      model_path=config.LLM_MODEL_PATH,
10     n_ctx=config.CONTEXT_LENGTH,
11     temperature=0.1 # 降低随机性, 确保结果精准
12 )
13
14 # 2. 加载Prompt模板 (直接复用, 无需修改)
15 # 数据问题识别Prompt
16 issue_detect_prompt = PromptTemplate(
17     input_variables=["data_sample"],
18     template="角色: 专业AI数据清洗Agent, 严谨高效, 仅输出结构化结果, 不添加多余解释。
19     任务: 识别以下数据样本中的所有问题, 包括但不限于重复值、缺失值、格式错误 (日期、数值、文本)、异常值, 按问题类型|涉及列名|具体描述的格式输出, 确保无遗漏。
20     数据样本: {data_sample}"
21 )
22

```

```
23 # 清洗指令生成Prompt
24 clean_code_prompt = PromptTemplate(
25     input_variables=["data_issues"],
26     template="角色：AI数据清洗Agent，精通pandas数据处理，生成的代码可直接运行，无需修
    改。
27     任务：根据以下识别出的数据问题，生成对应的pandas清洗代码，要求清洗后数据格式统一、无异
    常，代码末尾需返回清洗后的数据。
28     数据问题：{data_issues}"
29 )
30
31 # 清洗效果校验Prompt
32 check_effect_prompt = PromptTemplate(
33     input_variables=["data_issues", "cleaned_data_sample"],
34     template="角色：AI数据质量校验Agent，严格核对清洗效果，仅输出校验报告。
35     任务：对比原始数据问题和清洗后的数据，输出校验报告，包含清洗完成率、数据准确率、未解决问
    题（若有），若有未解决问题，补充优化代码。
36     原始数据问题：{data_issues}
37     清洗后数据样本：{cleaned_data_sample}"
38 )
39
40 # 3. 核心功能函数（无需修改）
41 def load_data(data_path):
42     """加载原始数据，支持CSV、Excel、MySQL数据库"""
43     if data_path.endswith(".csv"):
44         return pd.read_csv(data_path, encoding="utf-8")
45     elif data_path.endswith(".xlsx"):
46         return pd.read_excel(data_path)
47     elif "mysql://" in data_path:
48         # 适配MySQL数据库（示例：mysql://user:password@host:port/dbname?
49         table=table_name)
50         import pymysql
51         from sqlalchemy import create_engine
52         engine = create_engine(data_path)
53         return pd.read_sql_table(table_name=data_path.split("table=")[1],
54 con=engine)
55     else:
56         raise ValueError("仅支持CSV、Excel、MySQL格式数据")
57
58 def detect_data_issues(data):
59     """自动识别数据问题"""
60     # 取前100条样本识别问题（兼顾效率与准确性）
61     data_sample = data.head(100).to_string()
62     issues = llm(issue_detect_prompt.format(data_sample=data_sample))
63     return issues
64
65 def auto_clean_data(data, issues):
66     """自动执行数据清洗"""
```

```

65     # 生成清洗代码并执行
66     clean_code = llm(clean_code_prompt.format(data_issues=issues))
67     exec(clean_code, globals(), locals())
68     return locals()["cleaned_data"] # 返回清洗后的数据
69
70 def check_clean_effect(issues, cleaned_data):
71     """校验清洗效果，生成校验报告"""
72     cleaned_sample = cleaned_data.head(100).to_string()
73     check_report = llm(check_effect_prompt.format(
74         data_issues=issues,
75         cleaned_data_sample=cleaned_sample
76     ))
77     return check_report
78
79 # 4. 主函数（仅修改2处数据路径）
80 def main():
81     # 需修改1: 原始数据路径 (CSV/Excel/MySQL)
82     raw_data_path = f"{config.DATA_STORAGE_PATH}/raw_data.csv"
83     # 需修改2: 清洗后数据输出路径
84     output_data_path = f"{config.DATA_STORAGE_PATH}/cleaned_data.csv"
85
86     # 执行清洗流程（无需修改）
87     try:
88         data = load_data(raw_data_path)
89         print("数据加载成功，开始识别数据问题...")
90         issues = detect_data_issues(data)
91         print("数据问题识别完成，开始自动清洗...")
92         cleaned_data = auto_clean_data(data, issues)
93         print("数据清洗完成，开始校验效果...")
94         check_report = check_clean_effect(issues, cleaned_data)
95
96         # 保存清洗后的数据
97         cleaned_data.to_csv(output_data_path, index=False, encoding="utf-8")
98         print("\n=== 清洗完成 ===")
99         print(f"清洗后数据已保存至: {output_data_path}")
100        print("\n校验报告: ")
101        print(check_report)
102    except Exception as e:
103        print(f"执行失败，错误原因: {str(e)}")
104
105 if __name__ == "__main__":
106     main()

```

运行说明：1. 保存为clean_data_agent.py，与config.py放在同一目录；2. 运行前需修改main函数中2处路径（原始数据路径、清洗后数据输出路径），支持CSV、Excel、MySQL三种格式；3. 依赖pandas、langchain等包，需确保已安装；4. 数据量较大时，可修改config.py中的

BATCH_PROCESS_SIZE参数，避免内存不足；5. 运行后会自动生成清洗报告和清洗后的文件，可直接查看结果。

四、场景2：多模态数据自动化标注（annotation_agent.py）

```
1  # 多模态数据自动化标注完整代码 (annotation_agent.py)
2  import os
3  import base64
4  from langchain.llms import LlamaCpp
5  from langchain.prompts import PromptTemplate
6  from config import config
7  from PIL import Image
8  import speech_recognition as sr
9
10 # 1. 初始化大模型 (无需修改)
11 llm = LlamaCpp(
12     model_path=config.LLM_MODEL_PATH,
13     n_ctx=config.CONTEXT_LENGTH,
14     temperature=0.1
15 )
16
17 # 2. 加载Prompt模板 (直接复用, 无需修改)
18 # 文本标注Prompt
19 text_annotation_prompt = PromptTemplate(
20     input_variables=["text_data"],
21     template="角色：AI文本标注Agent，精准高效，严格按照标注规则执行，仅输出结构化标注结果。
22     标注规则：1. 情感分类：分为正面、负面、中性三类；2. 关键词提取：提取与核心问题、需求相关的3-5个关键词；3. 输出格式：文本内容|情感标签|关键词（用逗号分隔）。
23     任务：按照标注规则，对以下文本数据进行标注，无需多余解释，直接输出结果。
24     文本数据：{text_data}"
25 )
26
27 # 音频标注Prompt
28 audio_annotation_prompt = PromptTemplate(
29     input_variables=["audio_text"],
30     template="角色：AI音频标注Agent，已完成音频转写，仅进行关键信息提取和结构化输出。
31     标注规则：1. 提取关键信息：用户问题、需求、解决方案、客服回复重点；2. 输出格式：音频转写文本|用户问题|用户需求|解决方案。
32     任务：对以下音频转写文本进行标注，无需多余解释，直接输出结果。
33     音频转写文本：{audio_text}"
34 )
35
36 # 图像标注Prompt
37 image_annotation_prompt = PromptTemplate(
38     input_variables=["image_info", "image_category"],
```

```
39     template="角色：AI图像标注Agent，识别图像中的目标，进行分类和边界框标注，仅输出结
    构化结果。
40     标注规则：1. 目标分类：按照{image_category}进行分类；2. 边界框标注：输出目标的
    x1,y1,x2,y2坐标（左上角和右下角）；3. 输出格式：图像路径|目标分类|边界框坐标|置信度。
41     任务：对以下图像数据进行标注，无需多余解释，直接输出结果。
42     图像信息：{image_info}
43     图像分类规则：{image_category}"
44 )
45
46 # 3. 多模态数据处理辅助函数（无需修改）
47 def process_text_data(text_path):
48     """加载文本数据，返回文本列表"""
49     with open(text_path, "r", encoding="utf-8") as f:
50         return [line.strip() for line in f if line.strip()]
51
52 def process_audio_data(audio_path):
53     """将音频转写为文本"""
54     r = sr.Recognizer()
55     with sr.AudioFile(audio_path) as source:
56         audio = r.record(source)
57     try:
58         return r.recognize_google(audio, language="zh-CN")
59     except:
60         return "音频转写失败，请检查音频文件"
61
62 def process_image_data(image_path):
63     """获取图像基础信息，用于标注"""
64     img = Image.open(image_path)
65     width, height = img.size
66     return f"图像路径：{image_path}，尺寸：{width}x{height}，图像格式：
    {img.format}"
67
68 # 4. 核心标注函数（无需修改）
69 def text_annotation(text_data):
70     """文本数据标注"""
71     annotations = []
72     for text in text_data:
73         if not text:
74             continue
75         annotation = llm(text_annotation_prompt.format(text_data=text))
76         annotations.append(annotation)
77     return annotations
78
79 def audio_annotation(audio_path):
80     """音频数据标注"""
81     audio_text = process_audio_data(audio_path)
82     if "转写失败" in audio_text:
```

```

83         return [f"{audio_path}|转写失败|无|无|无"]
84     annotation = llm(audio_annotation_prompt.format(audio_text=audio_text))
85     return [f"{audio_path}|{annotation}"]
86
87 def image_annotation(image_path, image_category):
88     """图像数据标注"""
89     image_info = process_image_data(image_path)
90     annotation = llm(image_annotation_prompt.format(
91         image_info=image_info,
92         image_category=image_category
93     ))
94     return [annotation]
95
96 # 5. 主函数 (修改数据路径和标注规则即可)
97 def main():
98     # 需修改1: 标注类型 (text/audio/image)
99     annotation_type = "text"
100    # 需修改2: 数据路径 (文本/音频/图像文件夹或单个文件)
101    data_path = f"{config.DATA_STORAGE_PATH}/text_data.txt"
102    # 需修改3: 图像标注专用 (若为image类型, 填写分类规则; 其他类型无需修改)
103    image_category = "产品、背景、文字、logo"
104
105    # 执行标注流程 (无需修改)
106    try:
107        annotations = []
108        if annotation_type == "text":
109            text_data = process_text_data(data_path)
110            annotations = text_annotation(text_data)
111        elif annotation_type == "audio":
112            # 若为文件夹, 遍历所有音频文件
113            if os.path.isdir(data_path):
114                for file in os.listdir(data_path):
115                    if file.endswith((".wav", ".mp3")):
116                        audio_file = os.path.join(data_path, file)
117                        annotations.extend(audio_annotation(audio_file))
118            else:
119                annotations = audio_annotation(data_path)
120        elif annotation_type == "image":
121            # 若为文件夹, 遍历所有图像文件
122            if os.path.isdir(data_path):
123                for file in os.listdir(data_path):
124                    if file.endswith((".jpg", ".png", ".jpeg")):
125                        image_file = os.path.join(data_path, file)
126                        annotations.extend(image_annotation(image_file,
127image_category))
128            else:
129                annotations = image_annotation(data_path, image_category)

```



```

129         else:
130             raise ValueError("标注类型仅支持text、audio、image")
131
132         # 保存标注结果
133         output_path = f"
{config.DATA_STORAGE_PATH}/{annotation_type}_annotations.txt"
134         with open(output_path, "w", encoding="utf-8") as f:
135             f.write("\n".join(annotations))
136
137         print(f"标注完成！共标注{len(annotations)}条数据，结果已保存至：
{output_path}")
138     except Exception as e:
139         print(f"标注失败，错误原因：{str(e)}&#34;)&#xA;&#xA;if __name__ ==
&#34;__main__&#34;:&#xA;    main()

```

运行说明：1. 保存为annotation_agent.py，与config.py放在同一目录；2. 运行前需修改3处核心配置（标注类型、数据路径、图像分类规则，非图像类型可忽略第3点）；3. 依赖PIL、speech_recognition等包，需额外安装（pip install pillow speechrecognition）；4. 音频仅支持wav、mp3格式，图像仅支持jpg、png、jpeg格式，确保文件清晰无损坏；5. 数据路径可填写单个文件或文件夹，文件夹会自动遍历所有对应格式文件；6. 标注结果保存为txt文件，可直接用于大模型训练或后续分析。

五、场景3：数据合规审计+血缘追踪（compliance_agent.py）

```

1  # 数据合规审计+血缘追踪完整代码 (compliance_agent.py)
2  import pandas as pd
3  from langchain.llms import LlamaCpp
4  from langchain.prompts import PromptTemplate
5  from config import config
6
7  # 1. 初始化大模型（无需修改）
8  llm = LlamaCpp(
9      model_path=config.LLM_MODEL_PATH,
10     n_ctx=config.CONTEXT_LENGTH,
11     temperature=0.1
12 )
13
14 # 2. 加载Prompt模板（直接复用，无需修改）
15 # 敏感数据识别Prompt
16 sensitive_data_prompt = PromptTemplate(
17     input_variables=["data_sample"],
18     template="角色：AI合规审计Agent，熟悉行业合规规则，精准识别敏感数据，仅输出结构化
结果。
19     合规规则：敏感数据包括身份证号、手机号、银行卡号、邮箱、住址、企业机密信息等。

```



```
20 任务：识别以下数据中的敏感数据，按敏感数据类型|所在列|具体内容（脱敏处理）的格式输出，无
    需多余解释。
21 数据样本：{data_sample}"
22 )
23
24 # 数据血缘追踪Prompt
25 data_lineage_prompt = PromptTemplate(
26     input_variables=["data_info", "process_records"],
27     template="角色：AI数据血缘追踪Agent，自动梳理数据流转链路，仅输出结构化血缘图谱。
28 任务：根据以下数据的来源、处理记录，梳理数据全链路流转路径，输出数据来源→处理节点→处理
    方式→数据去向的结构化结果，无需多余解释。
29 数据信息：{data_info}
30 处理记录：{process_records}"
31 )
32
33 # 合规审计报告生成Prompt
34 compliance_report_prompt = PromptTemplate(
35     input_variables=["sensitive_data_result", "lineage_result"],
36     template="角色：AI合规审计Agent，根据敏感数据识别结果和血缘追踪结果，生成标准化合
    规审计报告。
37 任务：生成合规审计报告，包含报告标题、审计范围、敏感数据识别结果、血缘链路完整性、合规结
    论、整改建议（若有），格式规范，可直接用于监管提交。
38 敏感数据识别结果：{sensitive_data_result}
39 数据血缘追踪结果：{lineage_result}"
40 )
41
42 # 3. 核心功能函数（无需修改）
43 def load_data_for_compliance(data_path):
44     """加载合规审计所需数据"""
45     return pd.read_csv(data_path, encoding="utf-8")
46
47 def identify_sensitive_data(data):
48     """自动识别敏感数据"""
49     data_sample = data.head(200).to_string() # 取前200条样本，确保覆盖所有字段
50     sensitive_result =
51     llm(sensitive_data_prompt.format(data_sample=data_sample))
52     return sensitive_result
53
54 def track_data_lineage(data_info, process_records):
55     """自动梳理数据血缘链路"""
56     lineage_result = llm(data_lineage_prompt.format(
57         data_info=data_info,
58         process_records=process_records
59     ))
60     return lineage_result
61
62 def generate_compliance_report(sensitive_result, lineage_result):
```

```
62     """自动生成合规审计报告"""
63     report = llm(compliance_report_prompt.format(
64         sensitive_data_result=sensitive_result,
65         lineage_result=lineage_result
66     ))
67     return report
68
69 # 4. 主函数 (修改数据路径和处理记录即可)
70 def main():
71     # 需修改1: 待审计数据路径
72     data_path = f"{config.DATA_STORAGE_PATH}/to_audit_data.csv"
73     # 需修改2: 数据信息和处理记录 (填写数据来源、处理节点等)
74     data_info = "数据名称: 企业用户信息表, 数据来源: 业务系统, 数据用途: 用户管理与分
析"
75     process_records = "处理节点1: 数据采集 (业务系统同步), 处理方式: 批量导入; 处理节
点2: 数据清洗 (去重、脱敏), 处理方式: AI Agent自动处理; 处理节点3: 数据流转, 去向: 数据
分析部门、运营部门"
76
77     # 执行合规审计流程 (无需修改)
78     try:
79         data = load_data_for_compliance(data_path)
80         print("数据加载成功, 开始识别敏感数据...")
81         sensitive_result = identify_sensitive_data(data)
82         print("敏感数据识别完成, 开始梳理数据血缘...")
83         lineage_result = track_data_lineage(data_info, process_records)
84         print("数据血缘梳理完成, 开始生成合规审计报告...")
85         compliance_report = generate_compliance_report(sensitive_result,
lineage_result)
86
87         # 保存审计结果和报告
88         sensitive_output = f"{config.DATA_STORAGE_PATH}/敏感数据识别结果.txt"
89         lineage_output = f"{config.DATA_STORAGE_PATH}/数据血缘追踪结果.txt"
90         report_output = f"{config.DATA_STORAGE_PATH}/合规审计报告.txt"
91
92         with open(sensitive_output, "w", encoding="utf-8") as f:
93             f.write(sensitive_result)
94         with open(lineage_output, "w", encoding="utf-8") as f:
95             f.write(lineage_result)
96         with open(report_output, "w", encoding="utf-8") as f:
97             f.write(compliance_report)
98
99         print("\n=== 合规审计完成 ===")
100         print(f"敏感数据识别结果: {sensitive_output}")
101         print(f"数据血缘追踪结果: {lineage_output}")
102         print(f"合规审计报告: {report_output}")
103     except Exception as e:
104         print(f"合规审计失败, 错误原因: {str(e)}")
```

```
105
106     if __name__ == "__main__":
107         main()
```

运行说明：1. 保存为compliance_agent.py，与config.py放在同一目录；2. 运行前需修改2处核心内容（待审计数据路径、数据信息和处理记录），处理记录需如实填写数据流转节点，确保血缘追踪准确；3. 待审计数据仅支持CSV格式，若为其他格式可参考clean_data_agent.py的加载函数修改；4. 运行后会生成3个文件（敏感数据识别结果、数据血缘追踪结果、合规审计报告），报告可直接用于监管提交；5. 可在Prompt中添加行业专属敏感数据类型，适配不同行业合规要求。

六、补充说明

- 所有代码均依赖 config.py 配置文件，使用前请先修改 config.py 中的模型路径和数据存储路径；
- 运行前需确保已安装所有依赖包（安装指令见实战手册2.1节）；
- 每个代码文件可单独运行，无需关联其他文件（除config.py外），直接对应实战手册中的各个场景。